Wireless Environmental Monitoring System

Using Microchip PolarFire SoC Discovery Kit

Introduction

This document presents the development of a **Wireless Environmental Monitoring System** implemented using two Microchip PolarFire SoC Discovery Kits. Each board acts as an independent node within a wireless network — one configured as a **Transmitter Node (TX)** for sensing and transmission, and the other as a **Receiver Node (RX)** for data reception and display.

The purpose of this project is to demonstrate real-time environmental data acquisition, wireless communication, and embedded display interfacing using standard peripheral protocols (I2C and SPI). The **transmitter node (TX)** periodically measures temperature and humidity using the HDC2080 sensor and sends this data to the **receiver node (RX)** via the nRF24L01 RF module operating at 2.4 GHz. The receiver node displays the received data on an SPI-driven OLED display, achieving a complete sensor-to-display embedded link.

Note: To work with this project, you must first download and install **Microchip Libero SoC** and **SoftConsole**. Libero SoC is used for FPGA configuration and hardware design, while SoftConsole is used for embedded firmware development and debugging. After installation, you are required to **rewrite the entire project code** from scratch inside SoftConsole, implementing peripheral initialization, sensor data acquisition, wireless SPI communication, and OLED display control. This ensures you understand and replicate the system functionality on your own PolarFire SoC Discovery Kit setup.

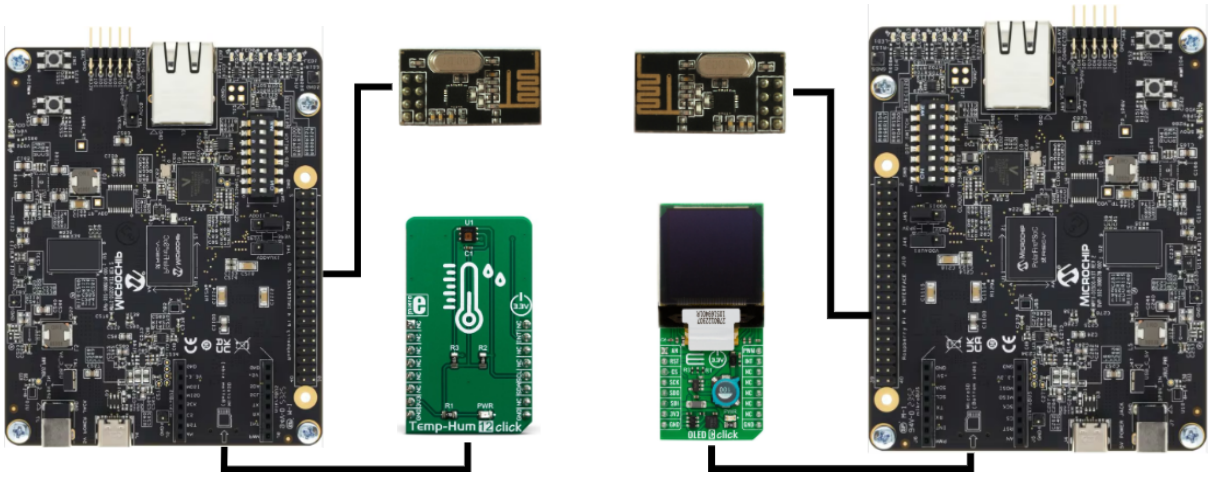


Figure 1: PolarFire SoC Discovery Kit Nodes — Transmitter (TX, left) and Receiver (RX, right).

Contents

1 Project Overview **3**

2 System Architecture **3**

3 Component Descriptions **4**
3.1 Microchip PolarFire SoC Discovery Kit 4
3.2 HDC2080 Temperature and Humidity Sensor 5
3.3 nRF24L01 Wireless Transceiver Module 5
3.4 OLED Display Module (SSD1351 Controller) 6

4 Block Diagram **7**

5 Communication Flow **7**

6 Software Design **8**
6.1 Pseudo-code (Transmitter and Receiver) 9
6.2 Node 1 (Transmitter - TX) Flow 11
6.3 Node 2 (Receiver - RX) Flow 11

7 Hardware Setup **11**

8 Implementation Stages **13**

9 Future Enhancements **13**

10 References **13**

Abstract

This report presents the design, implementation, and validation of a **wireless environmental monitoring system** using Microchip PolarFire SoC Discovery Kits. The system is composed of two nodes: a **transmitter node (TX)** that senses temperature and humidity, and a **receiver node (RX)** that displays the received environmental data. Both nodes communicate through the nRF24L01 RF module using the SPI protocol. The document details the system architecture, communication flow, hardware setup, software design, and implementation stages, along with possible extensions such as data logging and multi-node network expansion.

1. Project Overview

Environmental monitoring systems are essential for data acquisition and control in applications such as agriculture, smart buildings, and industrial automation. This project demonstrates a simple yet scalable design that combines temperature sensing, wireless communication, and data visualization using embedded systems.

2. System Architecture

The system consists of two main components, each implemented on a PolarFire SoC Discovery Kit with peripheral modules.

Node 1 – Transmitter (TX)

- PolarFire SoC Discovery Kit
- HDC2080 Temperature and Humidity Sensor (I2C interface)
- nRF24L01 RF Module (SPI interface)

Function: Periodically reads temperature and humidity values from the HDC2080 and transmits the data wirelessly.

Node 2 – Receiver (RX)

- PolarFire SoC Discovery Kit
- OLED Display (SPI interface)
- nRF24L01 RF Module (SPI interface)

Function: Receives wireless data packets and displays the measured temperature and humidity on the OLED.

3. Component Descriptions

This section describes each major component of the system and explains its internal structure and operating principles.

3.1. Microchip PolarFire SoC Discovery Kit

The PolarFire SoC Discovery Kit serves as the main controller for both nodes. It integrates a **RISC-V CPU subsystem** with programmable **FPGA fabric**, enabling hybrid hardware-software designs. Key internal elements include:

- **Microprocessor Subsystem (MSS)** – contains a quad-core 64-bit RISC-V CPU cluster (U54s) and a single management core (E51), handling application logic and peripheral control.
- **FPGA Fabric** – allows custom logic design and peripheral expansion using the Libero SoC toolchain.
- **Peripheral Interfaces** – SPI, I2C, UART, GPIO, and PWM for connecting sensors and communication modules.
- **Clocking and Power Units** – provide stable timing and energy-efficient operation.

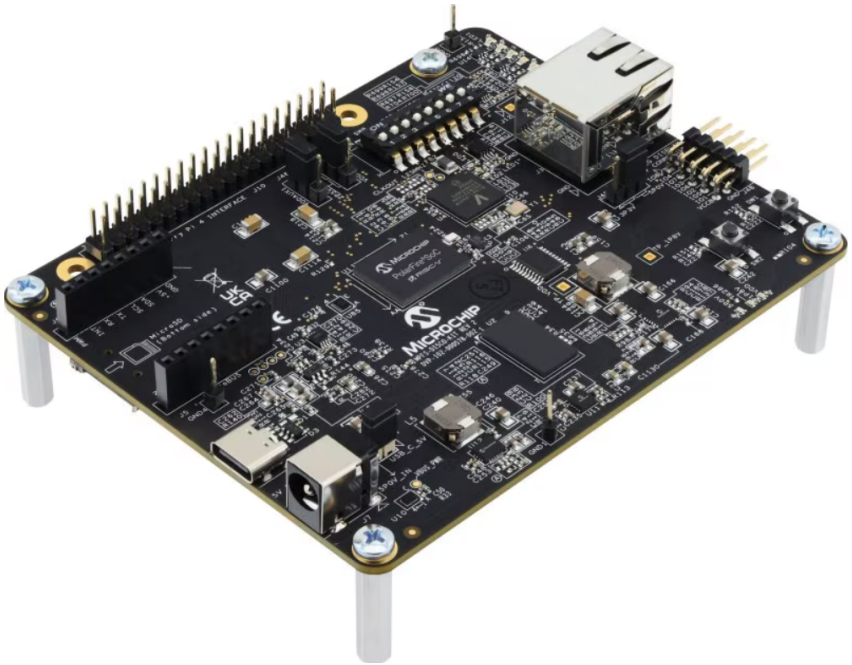


Figure 2: PolarFire SoC Discovery Kit

3.2. HDC2080 Temperature and Humidity Sensor

The HDC2080 is a low-power digital humidity and temperature sensor by Texas Instruments that communicates via the **I2C** protocol. Internally, it includes:

- **Capacitive Humidity Sensor** – measures relative humidity based on dielectric changes.
- **Bandgap Temperature Sensor** – provides accurate temperature readings from $-40°C$ to $+85°C$.
- **14-bit ADC** – converts analog sensor outputs into digital form.
- **Configuration Registers** – control measurement mode, interrupt thresholds, and data rate.

Its low power consumption and high accuracy make it ideal for battery-powered embedded systems.

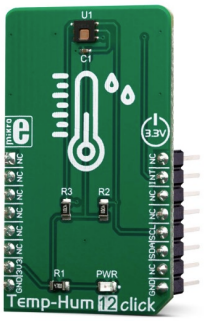


Figure 3: HDC2080 Sensor

3.3. nRF24L01 Wireless Transceiver Module

The nRF24L01 is a 2.4 GHz ISM-band transceiver by Nordic Semiconductor that uses the **SPI** protocol for MCU communication. Internally, the module integrates:

- **GFSK Modulator/Demodulator** – encodes data into a Gaussian Frequency-Shift Keyed signal.
- **Packet Handler** – manages preamble, address, CRC, and payload.
- **ShockBurst Engine** – handles automatic packet acknowledgment and retransmission.
- **Power Amplifier and LNA** – ensure stable RF transmission and reception, PA Gain 18dB, LNA Gain 20d.
- **2.4 GHz Antenna and front end Matching** – provides impedance matching for efficient wireless performance.

This chip supports up to 2 Mbps data rate and multiple communication channels, enabling fast and reliable wireless links.

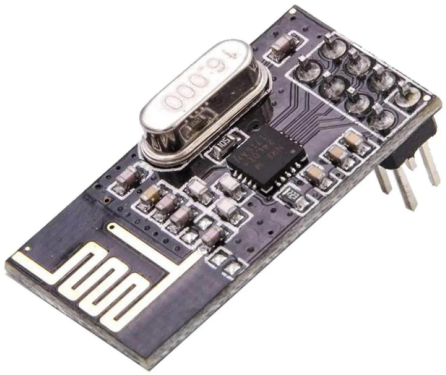


Figure 4: nRF24L01 Module

3.4. OLED Display Module (SSD1351 Controller)

The OLED display is based on the **SSD1351** controller, which interfaces via SPI. It offers bright visuals for data visualization and contains the following internal blocks:

- **Display RAM (GRAM)** – 128×128 memory for pixel color storage.
- **Timing Generator and Scan Driver** – refreshes pixels row by row.
- **Color Engine** – supports 16-bit or 18-bit color modes.
- **Command Decoder** – interprets incoming SPI commands (e.g., display ON/OFF, contrast, fill).

The OLED's high contrast and low latency make it suitable for real-time sensor data visualization.



Figure 5: OLED Display Module

4. Block Diagram

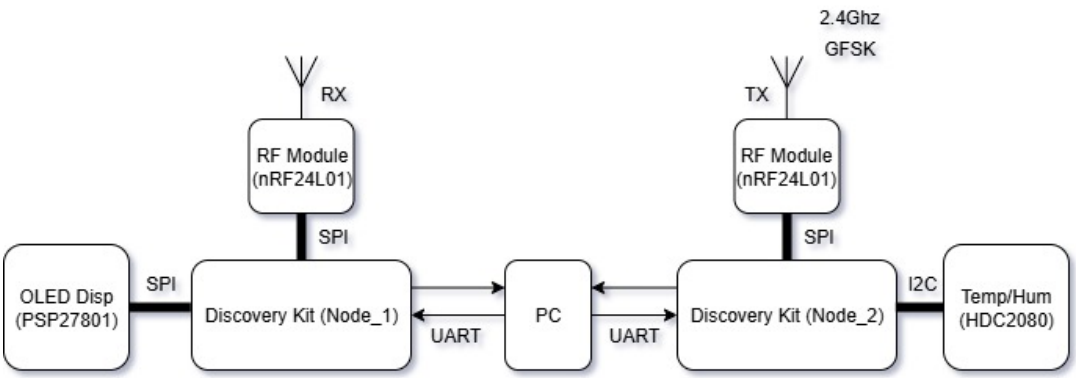


Figure 6: System-level block diagram of the wireless environmental monitoring setup

The block diagram illustrates the complete functional flow between the system's main components. It shows how the HDC2080 sensor measures temperature and humidity data, which is then processed and transmitted by the nRF24L01 module using SPI communication. The receiver node collects the transmitted data through its own nRF24L01 module and forwards it to the OLED display for visualization. Both transmitter and receiver are controlled by the PolarFire SoC Discovery Kits, which coordinate sensor readings, data transmission, and display updates.

5. Communication Flow

The data flow between nodes occurs over a 2.4 GHz wireless link using the nRF24L01 modules.

1. The transmitter node (TX) reads the temperature and humidity data from the HDC2080 sensor.
2. The data is packed into a structured message and transmitted via the nRF24L01 RF module using the SPI interface.
3. The receiver node (RX), configured in listening mode, receives the message.
4. The data is decoded and displayed on the OLED screen.

Both transmitter and receiver modules are configured with the following RF parameters:

- **Channel Number:** 76
- **Data Rate:** 1 Mbps
- **Packet Size:** 32 bytes
- **Output Power:** 0 dBm

To validate the wireless transmission and analyze the signal characteristics, an **Analog Devices ADALM-Pluto** software-defined radio (SDR) was used as a spectrum analyzer. The RF activity at 2.4 GHz was visualized using the **Satsagen** program, which provided real-time spectrum and power level monitoring of the transmitted nRF24L01 signal. This setup allowed confirmation of the transmitter's output power, and channel activity during live tests.

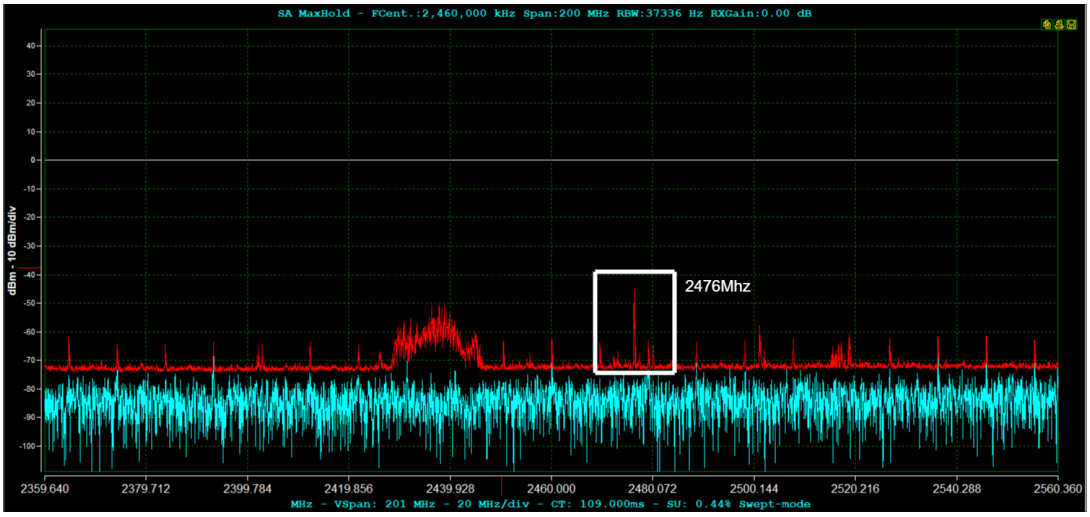


Figure 7: Wireless communication flow in channel 76 and spectrum visualization using ADALM-Pluto with Satsagen

6. Software Design

The software architecture is modular, separating sensor interfacing, communication, and display functionality.

Included Libraries in Implementation:

```
#include <stdint.h>
#include <string.h>
#include <stdio.h>

#include "mpfs_hal/mss_hal.h"
#include "drivers/mss/mss_spi/mss_spi.h"
#include "drivers/mss/mss_mmuart/mss_uart.h"
#include "drivers/mss/mss_gpio/mss_gpio.h"
#include "inc/uart_mapping.h"
#include "drivers/mss/mss_i2c/mss_i2c.h"
```

These libraries are available in the PolarFire SoC Bare-Metal Examples repository, under the `driver-examples` folder: Driver Examples – PolarFire SoC Bare-Metal

6.1. Pseudo-code (Transmitter and Receiver)

```
-- TRANSMITTER NODE (HDC2080 -> nRF24L01) --

setup():
    configure_clocks()
    init_uart(115200)           -- for logs
    init_gpio(nrf_ce_pin out)   -- CE control for nRF
    init_spi(spi0)              -- SPI for nRF
    init_i2c(i2c0)              -- I2C for HDC2080
    register_i2c_handler()
    probe_and_select_nrf_spi_mode_and_ss()
    init_hdc2080()              -- soft reset, verify IDs, configure
    configure_nrf_tx(channel=76, data_rate=1Mbps, output_power=0dBm,
                     addr=0xE7E7E7E7E7, payload_len=8)

main_loop():
    loop forever:
        trigger_hdc2080_measurement()
        wait_until_data_ready_with_timeout()
        if not ready:
            log("DRDY timeout; re-init sensor")
            hdc2080_soft_reset()
            hdc2080_init()
            continue

        read_raw_trh()            -- returns t_raw, rh_raw
        convert_to_centi_units(t_raw, rh_raw) -> t_centi, rh_centi
        log_uart( formatted t_centi, rh_centi )

        payload = pack_int32(t_centi) || pack_int32(rh_centi)   -- 8 bytes
        nrf_send_payload(payload, len=8)
        status = nrf_read_status()
        log_uart("SENT len=8 STATUS=0x%02X", status)

        delay(1000 ms)


-- RECEIVER NODE (nRF24L01 -> OLED) --

setup():
    configure_clocks()
    init_uart(115200)           -- for logs
    init_gpio(oled_en out, oled_dc out, nrf_ce out, nrf_irq in)
    init_spi(spi0)              -- SPI for OLED
    init_spi(spi1)              -- SPI for nRF
    init_oled_panel()           -- power up, init sequence, clear screen
    probe_and_select_nrf_spi_mode_and_ss_on_spi1()
```

```
    configure_nrf_rx(channel=76, data_rate=1Mbps, output_power=0dBm,
                     addr=0xE7E7E7E7E7, payload_len=8)
    set_nrf_to_listen_mode(ce_high)

main_loop():
    loop forever:
        if nrf_rx_fifo_not_empty():
            rx_payload = nrf_read_rx_payload(len=8)
            nrf_clear_irqs()

            unpack rx_payload -> t_centi, rh_centi
            t_str = format_centidegrees_to_string(t_centi)   -- e.g. "T: 23.45 C"
            rh_str = format_centidegrees_to_string(rh_centi) -- e.g. "RH: 56.78 %"

            log_uart("[RX] %s, %s", t_str, rh_str)

            oled_clear_area_or_fill_black()
            oled_draw_string(x=10,y=30, t_str)
            oled_draw_string(x=10,y=50, rh_str)

        delay(10 ms)


-- COMMON FUNCTIONS / NOTES --

probe_and_select_nrf_spi_mode_and_ss():
    for ss in {SS0, SS1}:
      for mode in {MODE0, MODE3}:
        configure_spi(ss, mode)
        write_reg(RF_CH, 76)
        read_back = read_reg(RF_CH)
        write_reg(CONFIG, PWR_UP | EN_CRC | (PRIM_RX optionally))
        cfg_back = read_reg(CONFIG)
        if read_back == 76 and cfg_back has PWR_UP:
            select this ss and mode and return success
    return failure -> log and halt

hdc2080 conversions:
    t_centi = (t_raw * 16500) / 65536 - 4000
    rh_centi = (rh_raw * 10000) / 65536

nrf_payload_format:
    payload[0..3] = little_endian_int32(t_centi)
    payload[4..7] = little_endian_int32(rh_centi)

error_handling:
    implement timeouts for I2C reads and DRDY polling
    if nRF repeatedly fails, attempt re-init sequence and log via UART
```

6.2. Node 1 (Transmitter - TX) Flow

1. Initialize peripherals (I2C, SPI, UART).
2. Read temperature and humidity from the HDC2080.
3. Format data into a transmission packet.
4. Transmit data through the nRF24L01 module.
5. Repeat at periodic intervals.

6.3. Node 2 (Receiver - RX) Flow

1. Initialize SPI and OLED display drivers.
2. Configure the nRF24L01 module in receive mode.
3. Wait for data packets from the transmitter.
4. Parse received data and display values on the OLED.

7. Hardware Setup

Each PolarFire SoC Discovery Kit communicates with peripheral modules through the MikroBUS adapter and the RPi connector.

Peripheral Connections (Node 1 – Transmitter/TX)

- **I2C0:** HDC2080 sensor (SDA, SCL, INT)
- **SPI0:** nRF24L01 (MOSI, MISO, SCK, CSN, CE, IRQ)
- **UART0:** Debug interface for temperature data output

Peripheral Connections (Node 2 – Receiver/RX)

- **SPI1:** nRF24L01 (MOSI, MISO, SCK, CSN, CE, IRQ)
- **SPI0:** OLED display (DC, EN, SDI, SCK, CS, RST, RW)
- **UART0:** Debug interface for received data

MikroBUS Socket and RPi Connector

The **MikroBUS Socket** provides a standardized interface for connecting "Click" boards such as the HDC2080, OLED, and nRF24L01. It exposes communication buses (SPI, I2C) and control lines (PWM, INT, RST, CS) to simplify hardware integration.

Additionally, the **RPi connector** on the PolarFire SoC Discovery Kit allows direct access to GPIO, SPI, I2C, and power pins, enabling custom wiring or expansion with Raspberry Pi-compatible peripherals. Together, the MikroBUS socket and RPi connector give flexibility in connecting sensors, actuators, and communication modules.

Hardware Connections

The hardware connections for the setup are shown below. **J5 and J6** are **MikroBUS connectors**, used for Click board modules like the HDC2080, OLED, or nRF24L01. **J10** is the **RPi connector**, providing access to GPIO, SPI, I2C, and power pins for custom peripheral connections.

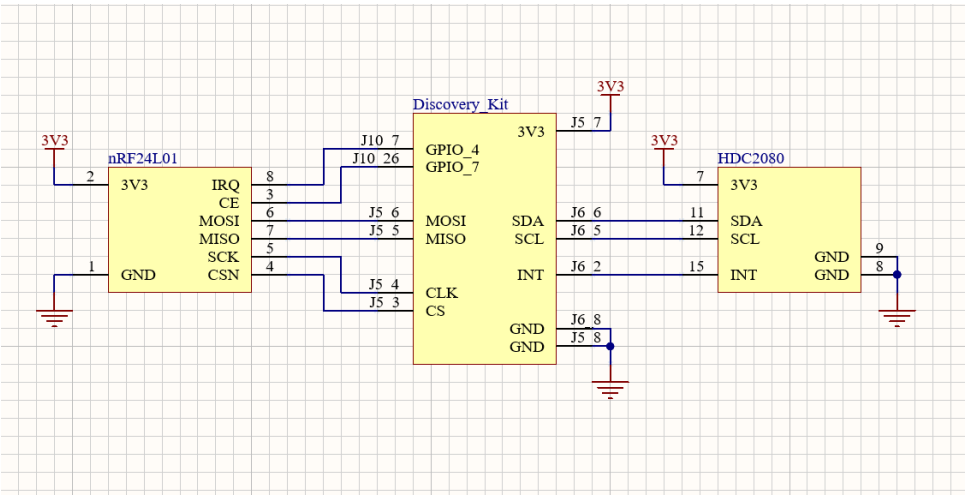


Figure 8: Transmitter node schematic, showing MikroBUS socket and RPi connector wiring

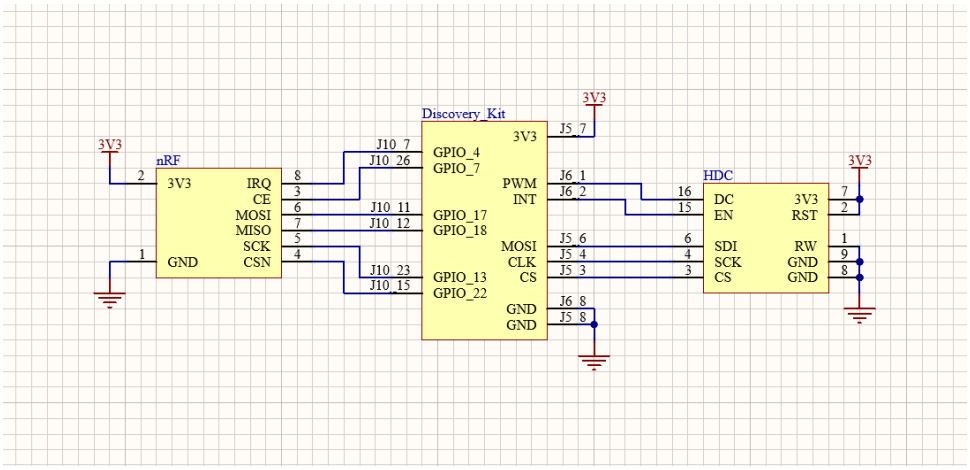


Figure 9: Receiver node schematic, showing MikroBUS socket and RPi connector wiring

For a more detailed schematic view of the PolarFire SoC Discovery Kit, visit: PolarFire SoC Discovery Kit Schematics (PDF). If any issues occur during data transmission, add a 10 µF capacitor between the 3.3 V and GND pins of the nRF24L01 module.

8. Implementation Stages

The development process was divided into incremental stages to simplify debugging and validation.

1. OLED Display Test

A simple program was written for Node 2 (RX) to print static or test messages on the OLED display using SPI communication.

2. Sensor Read Test

Node 1 (TX) read temperature and humidity data from the HDC2080 sensor via I2C and printed the results through UART for verification.

3. RF Communication Test

Basic nRF24L01 communication was implemented to transmit a static test message from Node 1 (TX) to Node 2 (RX). Successful reception was confirmed via UART.

4. End-to-End Integration

Sensor reading and RF transmission were combined on Node 1 (TX). Node 2 (RX) received the live data and displayed it on the OLED screen.

9. Future Enhancements

- Implement adaptive transmission intervals based on temperature changes.
- Add real-time clock (RTC) for timestamped data logging.
- Develop a PC-side visualization dashboard using UART or Ethernet.
- Extend wireless range and robustness with error-checking protocols.

10. References

- nRF24 RF24.h
- PolarFire SoC Discovery Kit Schematics
- Mikroelektronika OLED Display
- Mikroelektronika TempHum-12 Click
- nRF24L01 Datasheet
- GPIO1 Connections – PolarFire SoC Discovery Kit Reference Design
- Bare Metal Examples – PolarFire SoC